AI Object Detection

Summary:

This lesson teaches the arm to see. Lessons 6 to 8 turned a target position into joint angles, but the arm still cannot find a cup on the table. Here students run a pretrained object detector on a camera image and turn each detection into a bounding box the arm can act on. On Day 1, students learn what a bounding box is, how to measure the overlap of two boxes with IoU, and how to run a pretrained model that already knows 80 everyday object classes. On Day 2, students clean up the detector's output with a confidence threshold and non-maximum suppression, score it with precision and recall, then use the center of the best box to work out which way the base must turn. By the end of the lesson, students can run a detector on an image from the arm camera, filter its output, pick one target and calculate the base angle θ_B that points the arm at it. Lesson 10 adds the distance with a stereo camera.

Objective:

By the end of this lesson, students will be able to:

- Explain the difference between classifying an image and detecting objects in it
- Read a bounding box in corner format and in (x, y, width, height) format, and find its width, height, center and area
- Calculate the IoU (intersection over union) of two boxes and explain why 0.5 is a common match rule
- Describe what a pretrained model is, and what a detector returns for each object: a box, a label and a confidence score
- Run a pretrained detector on an image and read its output
- Apply a confidence threshold and non-maximum suppression, and explain the trade-off each one makes
- Calculate precision and recall from counts of true positives, false positives and false negatives
- Turn the center of a target box into a pixel offset and a base turn angle θ_B

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-3 (evaluate solutions against trade-offs), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data; Engaging in Argument from Evidence
- Crosscutting Concepts: Systems and System Models; Patterns

Vocabulary:

Object detection: Finding every object of interest in an image and giving each one a box, a label and a score.

Classification: Giving a whole image a single label, such as “cup”. It says what is in the image but not where.

Bounding box: A rectangle drawn around an object. It is stored as two corners, (x_min, y_min, x_max, y_max).

Image coordinates: Pixel positions measured from the top-left corner of the image. x grows to the right and y grows downward.

Center of a box: The point halfway between the corners, ((x_min + x_max) ÷ 2, (y_min + y_max) ÷ 2).

IoU (intersection over union): The overlap area of two boxes divided by the area they cover together. 0 means no overlap and 1 means identical boxes.

Pretrained model: A neural network that someone has already trained on a large set of labeled images, so we can use it without training it ourselves.

COCO: A large public set of labeled photos used to train detectors. It has 80 object classes, such as cup, bottle, scissors and apple.

Confidence score: A number from 0 to 1 that says how sure the model is about a detection. A higher score means more confident, not guaranteed correct.

Confidence threshold: The lowest score we accept. Detections below it are thrown away.

Non-maximum suppression (NMS): A clean-up step that keeps the best box for each object and removes overlapping duplicates.

True positive (TP): A detection that matches a real object: the right label and an IoU of 0.5 or more with the true box.

False positive (FP): A detection with no real object behind it, or with the wrong label.

False negative (FN): A real object that the model missed.

Precision: Of everything the model found, the fraction that was right: TP ÷ (TP + FP).

Recall: Of everything that was really there, the fraction the model found: TP ÷ (TP + FN).

Pixel offset: How many pixels the box center sits to the right (dx) and below (dy) the center of the image.

Focal length in pixels (f): A camera number that links pixels to angles: f = (W ÷ 2) ÷ tan(HFOV ÷ 2), where W is the image width and HFOV is the horizontal field of view.

Resources & Materials:

- Lesson 9 slideshow, Days 1 and 2 (28 slides; the slide numbers in the Procedures are from this slideshow)
- 10 to 15 photos taken with the arm camera of a tabletop with everyday objects (cup, bottle, scissors, apple, banana, book, cell phone). Include some photos with two similar objects close together, and some with nothing the model knows

- Computers with Python, the ultralytics and opencv-python packages, and the helper file detect_helper.py (code in the Appendix)
- Internet access the first time the model runs, so it can download the small pretrained model (yolov8n.pt). To run offline, download it once before class and keep it in the student folder
- Rulers and graph paper for the IoU activity, and calculators
- The assembled arm and its camera from the earlier lessons, to take live photos (optional)

Teacher note: the box math in detect_helper.py and the Day 2 script were tested with the numbers in this plan. run_detector downloads and runs the pretrained model, so run it once on a lab computer before class. The examples use a 640 × 480 image and a 60° horizontal field of view (HFOV). For the arm, use the HFOV of your camera from its data sheet.

Procedures:

Day 1: Bounding Boxes and a Pretrained Detector

- **Opening (5 min)** — Show the title slide, "Where Lessons 6 to 8 Left Off" and "From Camera to Base Angle" (slides 1–3). Inverse kinematics turned the target (301.78, 301.78, 176.78) into θB = 45°, θS = 45°, θE = −45°, but the arm still cannot find a cup on the table. Show an arm-camera photo of a tabletop and ask students to point at the cup. Ask: what did you work out to do that? Collect two answers: what it is and where it is. A detector answers both with a box. Walk through the five steps on slide 3: photo, boxes, clean-up, pick one target, turn angle. Day 1 covers steps 1 and 2; Day 2 covers steps 3 to 5.
- **One label or many boxes (5 min)** — Show "Classification, Localization, Detection" (slide 4). Classification gives the whole image one label ("cup"). Localization draws one box around one object. Detection does this for every object and gives each a label and a score. Ask students what a detector must output for the three-object picture: three boxes, three labels, three scores.
- **Image coordinates (4 min)** — Show "Pixels Have Their Own Axes" (slide 5). The origin (0, 0) is the top-left corner, x grows to the right and y grows downward. Note the change of habit: in the arm lessons, up was y or z, but in an image a larger y means lower in the picture. Common mistake: reading a larger y as "higher in the picture".
- **Bounding boxes (7 min)** — Show "Two Ways to Write a Box" (slide 6): corner format (x_min, y_min, x_max, y_max) and (x, y, width, height). Work through the box (120, 80, 360, 320) on "Worked Example: Box Measurements" (slide 7): width = 360 − 120 = 240, height = 320 − 80 = 240, center = (240, 200), area = 240 × 240 = 57,600 px². The two formats hold the same information, so (120, 80, 240, 240) is the same box. The detector in this lesson uses corner format.
- **Try It Yourself #1 (5 min)** — Students find the width, height, center and area of the box (50, 60, 250, 160) and write it as (x, y, width, height) (slide 8). Reveal the solution: width 200, height 100, center (150, 110), area 20,000 px², and (50, 60, 200, 100) in the other format.
- **IoU: how much do two boxes overlap? (9 min)** — Show "Intersection over Union" (slide 9). To compare a model's box with a true box we need one number for overlap: IoU = overlap area ÷ combined area. 0 means the boxes do not touch, 1 means they are identical, and 0.5 or more usually counts as a match. Teach the overlap recipe: the overlap box takes the larger of the two minimums and the smaller of the two maximums. Work "Worked Example: IoU" (slide 10) with squares drawn on graph paper: A = (100, 100, 300, 300) and B = (200, 200, 400, 400). The

overlap is (200, 200, 300, 300), which is 100 × 100 = 10,000 px². Each box is 40,000, so the combined area is 40,000 + 40,000 − 10,000 = 70,000 and IoU = 0.14, which is not a match. Common mistake: adding the two areas and forgetting to subtract the overlap, which counts the overlap twice.

- **Try It Yourself #2 (5 min)** — Students find the IoU of A = (0, 0, 100, 100) and B = (0, 0, 100, 50) (slide 11). Reveal the solution: B sits completely inside A, so the overlap is B's area, 5,000; the combined area is A's area, 10,000; IoU = 0.50. Bonus: the IoU of two identical boxes is 1.
- **What is a pretrained model? (7 min)** — Show "Someone Already Taught It" (slide 12). A pretrained detector has already learned from hundreds of thousands of labeled photos, so we do not train anything today. Its input is an image. Its output is a list, and each item has a box, a label and a confidence score. The model knows COCO's 80 classes (cup, bottle, scissors, apple and so on) and nothing else. Ask: what will it do with an object that is not one of the 80? Point out that its training photos were not taken by our arm camera, so some objects may be harder for it.
- **Check it with code (10 min)** — Show "Check It with Code" (slide 13). Students run `run_detector` from detect_helper.py on three arm-camera photos, print each label, score and box, and save a picture with `draw_detections`. Each student finds one mistake the model made (a wrong label, a missed object or a box on nothing) and writes it down for Day 2. Results depend on the photo, so there is no fixed output to compare against.
- **Recap (3 min)** — Show "Day 1 Recap" (slide 14): the box, IoU, the pretrained model and the three things a detector returns. Preview Day 2: the model's output is messy, so we clean it up and then use it to point the arm at a target.

Day 2: Cleaning Up Detections and Pointing the Arm

- **Recap (3 min)** — Show "Cleaning Up and Pointing the Arm" (slide 15). Quick-fire questions: Which way does y grow in an image? (Down.) What is the IoU of two boxes that do not touch? (0.) What three things does a detector return for each object? (Box, label, confidence score.) Ask two or three students to share the mistake they found on Day 1.
- **How sure is the model? (5 min)** — Show "How Sure Is the Model?" (slide 16). Every detection has a score from 0 to 1. We choose a threshold and throw away anything below it. For the scores 0.92, 0.85, 0.40 and 0.15, a threshold of 0.9 keeps one detection, 0.5 keeps two and 0.3 keeps three. Too low a threshold gives false alarms; too high a threshold misses real objects.
- **Lab: threshold sweep (8 min)** — Show "Lab: Threshold Sweep" (slide 17). Students run the detector on one photo with conf = 0.10, 0.25, 0.50 and 0.80 and fill in the table: how many boxes each setting gives and how many of them are right. Students pick the setting they would trust to guide a robot and defend it in one sentence.
- **Duplicate boxes and NMS (8 min)** — Show "Too Many Boxes" (slide 18). A detector often draws several boxes on the same object. Non-maximum suppression (NMS) fixes this: sort the boxes by score, keep the best one, remove every lower-scoring box with the same label that overlaps it by more than 0.5 IoU, then repeat with the next box that is left. YOLO already does this inside the model; we do it by hand to see how it works. Work "NMS Example" (slide 19): A = (100, 100, 300, 300) with score 0.90, B = (120, 110, 320, 310) with 0.70 and C = (500, 200, 600, 300) with 0.80. Keep A. The overlap of A and B is 180 × 190 = 34,200 and their union is 40,000 + 40,000 −

34,200 = 45,800, so IoU(A, B) = 0.75 and B is removed. C does not touch A, so it stays. The result is A and C: two boxes for two cups.

- **Try It Yourself #3 (5 min)** — Students run NMS with a 0.5 limit on A = (0, 0, 100, 100) with 0.90, B = (10, 0, 110, 100) with 0.80 and C = (300, 300, 400, 400) with 0.70 (slide 20). Reveal the solution: the overlap of A and B is 90 × 100 = 9,000 and their union is 11,000, so IoU(A, B) = 0.82 and B is removed. C is far away, so it stays. Kept: A and C.
- **Clean the list, then count (3 min)** — Show "Clean the List, Then Count" (slide 21). Put the steps together on one list: drop the 0.30 cup because it is below 0.5, remove the 0.62 cup with NMS because it is the same cup as the 0.91 one, and keep the rest. The result is 2 cups and 1 bottle.
- **Precision and recall (6 min)** — Show "Precision and Recall" (slide 22). Compare each detection with a hand-drawn true box. A detection with the right label and an IoU of 0.5 or more is a true positive (TP), a box on nothing or with the wrong label is a false positive (FP), and a real object that nothing matched is a false negative (FN). Precision = TP ÷ (TP + FP) and recall = TP ÷ (TP + FN). Example: 8 boxes drawn, 6 correct, 10 real objects, so precision = 6 ÷ 8 = 0.75 and recall = 6 ÷ 10 = 0.60. Discuss: for a gripper, which is worse, grabbing at nothing (a false positive) or never seeing the cup (a false negative)? Accept either answer if the student argues from the arm's behavior.
- **Try It Yourself #4 (4 min)** — The detector draws 5 boxes, 4 are correct, and the photo has 8 objects (slide 23). Reveal the solution: TP = 4, FP = 1, FN = 4, so precision = 4 ÷ 5 = 0.80 and recall = 4 ÷ 8 = 0.50. It is mostly right when it speaks, but it misses half the objects.
- **From box to turn angle (7 min)** — Show "From Box to Turn Angle" (slide 24). Four steps: the box center cx = (x_min + x_max) ÷ 2; the offset from the image center dx = cx − W ÷ 2; the camera's focal length in pixels f = (W ÷ 2) ÷ tan(HFOV ÷ 2); and the turn angle θ = atan(dx ÷ f). The base angle θB counts counter-clockwise from above, so a target on the right (dx > 0) means a negative turn. This assumes the camera looks along the arm's x axis; Lesson 10 finds the exact angle. Work "Turn Angle Example" (slide 25): W = 640 px, HFOV = 60° and the box (400, 150, 500, 350). cx = 450, dx = 130 px, f = 320 ÷ tan 30° = 554.3 px and θ = atan(130 ÷ 554.3) = 13.2°. The target is to the right, so θB = −13.2°.
- **Try It Yourself #5 (4 min)** — Same camera, box (60, 100, 180, 300) (slide 26). Reveal the solution: cx = 120, dx = −200 px and θ = atan(200 ÷ 554.3) = 19.8°. The target is to the left, so θB = +19.8°.
- **Check it with code (5 min)** — Show "The Whole Pipeline in Code" (slide 27) and run the Day 2 script in the Appendix: detect, filter by score, apply NMS, pick the best cup, then print its center, its pixel offset and the base turn. Students feed in the worked example boxes and check that they get −13.2° and +19.8°: right of center must give a negative θB.
- **Wrap-up (2 min)** — Show "What We Can Now Do" (slide 28): read a box and measure overlap with IoU; run a pretrained detector; filter by confidence and remove duplicates with NMS; score with precision and recall; turn the base toward a target. Preview Lesson 10: one camera gives the direction but not the distance. A stereo camera adds the depth.

Assessment:

Students complete the Try It Yourself problems (#1–#5) and check each answer against the solution on the slide. They can check #5 with the code on "The Whole Pipeline in Code" slide.

Use these questions as an exit ticket or short quiz. The answers are in red.

- In one sentence, what does detection do that classification does not?
  Answer: Detection finds every object and says where each one is (a box), while classification gives the whole image a single label.
- In an image, where is the origin (0, 0), and which way does y grow?
  Answer: The origin is the top-left corner, and y grows downward.
- Find the width, height, center and area of the box (100, 40, 260, 200).
  Answer: Width 160, height 160, center (180, 120), area 25,600 px^2.
- Write the box (50, 60, 250, 160) as (x, y, width, height).
  Answer: (50, 60, 200, 100)
- Find the IoU of A = (0, 0, 200, 200) and B = (100, 0, 300, 200).
  Answer: The overlap is 100 × 200 = 20,000 and the combined area is 40,000 + 40,000 − 20,000 = 60,000, so IoU = 0.33. That is less than 0.5, so they do not count as a match.
- True or false: a pretrained COCO detector can find any object you point the camera at.
  Answer: False. It only knows the 80 classes it was trained on.
- A detector gives scores 0.92, 0.85, 0.40 and 0.15. Which detections survive a threshold of 0.5? What happens if you raise it to 0.9?
  Answer: 0.92 and 0.85 survive. At 0.9 only 0.92 survives, so there are fewer false alarms but a greater chance of missing a real object.
- Two boxes with the same label have an IoU of 0.82. What does NMS do with them?
  Answer: It keeps the box with the higher score and removes the other one, because they are probably the same object.
- A detector gives TP = 8, FP = 2 and FN = 4. Find its precision and recall.
  Answer: Precision = 8 ÷ 10 = 0.80 and recall = 8 ÷ 12 = 0.67.
- In a 640 × 480 image with a 60° horizontal field of view, a target box is (400, 200, 500, 300). Which way should the base turn, and by about how much?
  Answer: The box center (450, 250) is 130 px right of the image center, so the angle is atan(130 ÷ 554.3) = 13.2°. The target is to the right, so θ_B is about −13.2°.

Appendix: detect_helper.py

Save this code as detect_helper.py in the same folder as the student code. Install the packages first with `pip install ultralytics opencv-python`. The box math was tested; run run_detector once on a lab computer before class.

```
"""Helper functions for the AI Object Detection lesson.
Boxes are (x_min, y_min, x_max, y_max) in pixels. The origin (0, 0) is the
TOP-LEFT corner of the image, x grows to the right and y grows DOWNWARD.
"""
import math


def box_size(box):
```

```
    """Width and height of a box."""
    x1, y1, x2, y2 = box
    return x2 - x1, y2 - y1


def box_area(box):
    w, h = box_size(box)
    return max(w, 0) * max(h, 0)


def box_center(box):
    """Center point (cx, cy) of a box."""
    x1, y1, x2, y2 = box
    return (x1 + x2) / 2, (y1 + y2) / 2


def xyxy_to_xywh(box):
    """Corner format -> (x_min, y_min, width, height)."""
    x1, y1, x2, y2 = box
    return x1, y1, x2 - x1, y2 - y1


def xywh_to_xyxy(box):
    """(x_min, y_min, width, height) -> corner format."""
    x, y, w, h = box
    return x, y, x + w, y + h


def iou(a, b):
    """Intersection over Union of two boxes (0 = no overlap, 1 = identical)."""
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(ix2 - ix1, 0) * max(iy2 - iy1, 0)
    union = box_area(a) + box_area(b) - inter
    return inter / union if union > 0 else 0.0


def filter_by_score(detections, threshold):
    """Keep detections whose score is at least threshold.
    Each detection is a dict:
    {"box": (x1, y1, x2, y2), "label": str, "score": float}
    """
    return [d for d in detections if d["score"] >= threshold]


def nms(detections, iou_threshold=0.5):
    """Non-maximum suppression. For each label, keep the best box and drop
    any lower-scoring box that overlaps it by more than iou_threshold."""
    kept = []
    for d in sorted(detections, key=lambda d: d["score"], reverse=True):
        clash = any(k["label"] == d["label"]
                    and iou(k["box"], d["box"]) > iou_threshold
                    for k in kept)
        if not clash:
            kept.append(d)
    return kept
```

```
def precision_recall(tp, fp, fn):
    """Precision = TP / (TP + FP). Recall = TP / (TP + FN)."""
    p = tp / (tp + fp) if tp + fp else 0.0
    r = tp / (tp + fn) if tp + fn else 0.0
    return p, r


def pixel_offset(box, image_w, image_h):
    """How far the box center is from the image center, in pixels.
    Positive dx = target is to the RIGHT. Positive dy = target is BELOW."""
    cx, cy = box_center(box)
    return cx - image_w / 2, cy - image_h / 2


def turn_angle(dx, image_w, hfov_deg):
    """Angle (degrees) between the camera's forward direction and a pixel dx
    pixels from the image center. Pinhole model: tan(angle) = dx / f."""
    f = (image_w / 2) / math.tan(math.radians(hfov_deg) / 2)  # in pixels
    return math.degrees(math.atan(dx / f))


def pick_target(detections, label):
    """The highest-scoring detection with the given label, or None."""
    matches = [d for d in detections if d["label"] == label]
    return max(matches, key=lambda d: d["score"]) if matches else None


def run_detector(image_path, model_name="yolov8n.pt", conf=0.25):
    """Run a pretrained YOLO model (trained on COCO's 80 classes) on one image.
    Returns a list of {"box", "label", "score"} dicts.
    Needs: pip install ultralytics
    """
    from ultralytics import YOLO
    model = YOLO(model_name)  # downloads the weights the first time
    result = model(image_path, conf=conf, verbose=False)[0]
    out = []
    for b in result.boxes:
        x1, y1, x2, y2 = [float(v) for v in b.xyxy[0]]
        out.append({"box": (x1, y1, x2, y2),
                    "label": result.names[int(b.cls[0])],
                    "score": float(b.conf[0])})
    return out


def draw_detections(image_path, detections, out_path="detections.png"):
    """Draw each box and its label + score on the image and save it."""
    import cv2
    img = cv2.imread(image_path)
    for d in detections:
        x1, y1, x2, y2 = [int(v) for v in d["box"]]
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 200, 0), 2)
        text = f'{d["label"]} {d["score"]:.2f}'
        cv2.putText(img, text, (x1, max(y1 - 6, 12)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 200, 0), 1)
    cv2.imwrite(out_path, img)
    return out_path


if __name__ == "__main__":
```

```
    A, B = (100, 100, 300, 300), (200, 200, 400, 400)
    print(round(iou(A, B), 4))  # expected: 0.1429
```

Day 2 “Check It with Code” script. Students change the image name, the class and the thresholds. When the best cup box is (400, 150, 500, 350), it prints a center of (450.0, 250.0), an offset of 130.0 10.0 and a base turn of −13.2 degrees.

```
from detect_helper import *

img = "table.png"                  # an arm-camera photo, 640 x 480
W, H, HFOV = 640, 480, 60          # image size and horizontal field of view

dets = run_detector(img, conf=0.10)     # 1-2: photo and boxes, start low
dets = filter_by_score(dets, 0.50)      # 3: drop low scores
dets = nms(dets, 0.5)                   # 3: remove duplicate boxes
target = pick_target(dets, "cup")       # 4: the best cup, or None

if target is None:
    print("No cup found")
else:
    print("center:", box_center(target["box"]))
    dx, dy = pixel_offset(target["box"], W, H)
    print("offset:", dx, dy)
    # target on the right -> negative theta_B (it counts counter-clockwise)
    print("base turn (deg):", round(-turn_angle(dx, W, HFOV), 1))   # 5
```